

TP 5B

Déploiement du Pipeline IoT – Docker Compose

Mosquitto · InfluxDB · Node-RED · Grafana

Durée : 4 heures | Niveau : L2/L3 Informatique

Prérequis : TP5A validé – avoir publié des messages MQTT depuis Wokwi

Objectif pédagogique

Ce TP déploie le pipeline IoT complet que vous avez schématisé en TP5A.

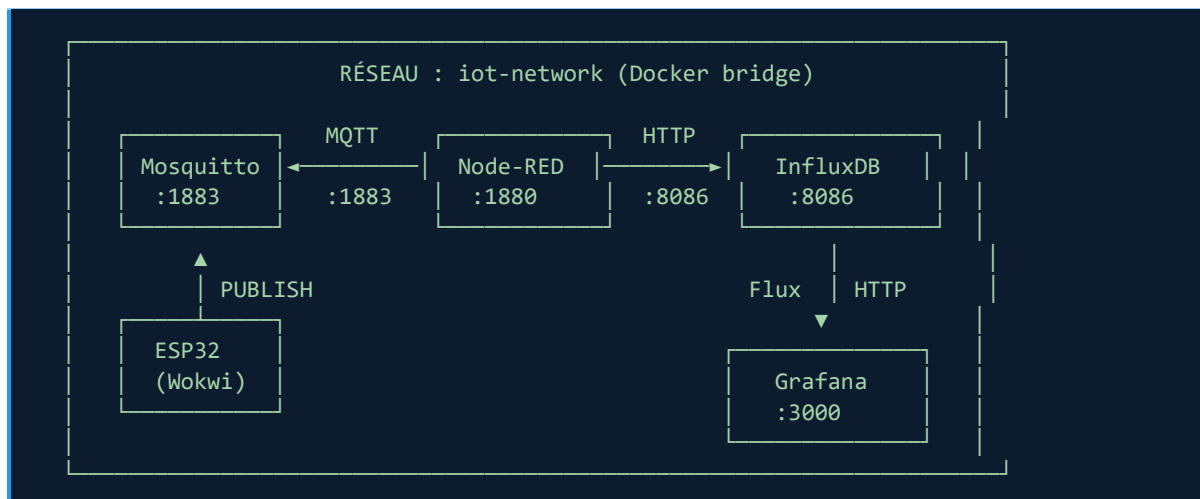
À l'issue de cette séance, chaque binôme aura :

- Installé et lancé Docker Desktop / Docker Engine sur la machine de TP
- Déployé Mosquitto, InfluxDB, Node-RED et Grafana via docker-compose.yml
- Configuré Node-RED pour souscrire à Mosquitto et insérer dans InfluxDB
- Visualisé les données capteurs en temps réel dans Grafana
- Configuré une alerte Grafana sur dépassement de seuil
- Connecté l'ESP32 Wokwi du TP5A à ce pipeline local (en option)

Ce TP constitue la base technique du projet ECUE partagé L2/L3

1. Architecture cible

Le pipeline que vous allez déployer est identique à celui que vous utiliserez pour votre projet ECUE de fin de semestre :



Service	Rôle	Port	Image Docker
Mosquitto	Broker MQTT – reçoit et redistribue les messages	1883	eclipse-mosquitto:2.0
InfluxDB	Base time-series – stocke les mesures horodatées	8086	influxdb:2.7
Node-RED	ETL – souscrit MQTT, transforme, insère dans InfluxDB	1880	nodered/node-red:latest
Grafana	Dashboard temps réel + alertes configurables	3000	grafana/grafana:latest

2. Prérequis et installation

2.1 Docker

Système	Installation
Windows 10/11	Docker Desktop – https://www.docker.com/products/docker-desktop/
macOS	Docker Desktop (M1/M2 supporté)
Ubuntu / Debian	<code>sudo apt install docker.io docker-compose-plugin</code>
Salle de TP (déjà installé)	Vérifier : <code>docker --version</code> && <code>docker compose version</code>

Vérification de l'installation :

```
$ docker --version
Docker version 24.x.x, build xxxxxxxx

$ docker compose version
Docker Compose version v2.x.x

$ docker run hello-world
Hello from Docker! ← Si ce message apparaît, tout est opérationnel.
```

2.2 Structure des fichiers du projet

Créez l'arborescence suivante sur votre machine de TP :

```
iot-pipeline/
├── docker-compose.yml      ← orchestration principale
├── mosquitto/
│   └── config/
│       └── mosquitto.conf  ← configuration du broker
├── influxdb/
│   └── .env                ← variables d'environnement InfluxDB
├── nodered/
│   └── flows/              ← dossier des flux Node-RED (auto-généré)
└── grafana/
    └── provisioning/       ← datasources et dashboards pré-configurés
```

3. Partie A – docker-compose.yml et premier démarrage (1h)

3.1 Fichier docker-compose.yml

Créez le fichier `iot-pipeline/docker-compose.yml` avec le contenu suivant :

```
#version: '3.8'

# — Réseau interne isolé —
networks:
  iot-network:
    driver: bridge

# — Volumes persistants —
volumes:
  influxdb-data:
  grafana-data:

services:

# — 1. Broker MQTT —
mosquitto:
  image: eclipse-mosquitto:2.0
  container_name: mosquitto
  restart: unless-stopped
  ports:
    - "1883:1883" # MQTT TCP
    - "9001:9001" # MQTT WebSocket (optionnel)
  volumes:
    - ./mosquitto/config/mosquitto.conf:/mosquitto/config/mosquitto.conf
  networks:
    - iot-network

# — 2. Base de données time-series —
influxdb:
  image: influxdb:2.7
  container_name: influxdb
  restart: unless-stopped
  ports:
    - "8086:8086"
  env_file:
    - ./influxdb/.env
  volumes:
    - influxdb-data:/var/lib/influxdb2
  networks:
    - iot-network

# — 3. Orchestration ETL —
node-red:
  image: nodered/node-red:latest
  container_name: node-red
  restart: unless-stopped
  ports:
    - "1880:1880"
  volumes:
    - ./nodered/flows:/data
  depends_on:
    - mosquitto
    - influxdb
  networks:
    - iot-network
```

```
# — 4. Dashboard —————
grafana:
  image: grafana/grafana:latest
  container_name: grafana
  restart: unless-stopped
  ports:
    - "3000:3000"
  environment:
    - GF_SECURITY_ADMIN_PASSWORD=ufhb2025
    - GF_USERS_ALLOW_SIGN_UP=false
  volumes:
    - grafana-data:/var/lib/grafana
  depends_on:
    - influxdb
  networks:
    - iot-network
```

3.2 Configuration Mosquitto

Créez le fichier `mosquitto/config/mosquitto.conf` :

```
# mosquitto.conf - configuration minimale pour ce TP
listener 1883
listener 9001
protocol websockets

# Autoriser les connexions anonymes (TP uniquement - à sécuriser en production)
allow_anonymous true

# Logging
log_type all
log_dest stdout
```

3.3 Variables InfluxDB

Créez le fichier `influxdb/.env` :

```
DOCKER_INFLUXDB_INIT_MODE=setup
DOCKER_INFLUXDB_INIT_USERNAME=admin
DOCKER_INFLUXDB_INIT_PASSWORD=ufhb2025admin
DOCKER_INFLUXDB_INIT_ORG=ufhb
DOCKER_INFLUXDB_INIT_BUCKET=capteurs_iot
DOCKER_INFLUXDB_INIT_RETENTION=30d
DOCKER_INFLUXDB_INIT_ADMIN_TOKEN=mon-token-secret-ufhb-2025
```

3.4 Premier démarrage

```
# Depuis le dossier iot-pipeline/
$ docker compose up -d

# Vérifier que les 4 conteneurs sont running
$ docker compose ps

# Résultat attendu :
```

```

NAME          STATUS    PORTS
mosquitto     running  0.0.0.0:1883->1883/tcp
influxdb      running  0.0.0.0:8086->8086/tcp
node-red      running  0.0.0.0:1880->1880/tcp
grafana       running  0.0.0.0:3000->3000/tcp

```

```

# Logs en temps réel (Ctrl+C pour quitter)
$ docker compose logs -f

```

Interfaces accessibles après démarrage

- InfluxDB : <http://localhost:8086> (login: admin / ufhb2025admin)
- Node-RED : <http://localhost:1880>
- Grafana : <http://localhost:3000> (login: admin / ufhb2025)

Questions de compréhension – Partie A

1. Quel est le rôle de la directive `depends_on` dans `docker-compose.yml` ?
2. Pourquoi utilise-t-on des volumes nommés (`influxdb-data`, `grafana-data`) ?
3. Que signifie `restart: unless-stopped` ? Dans quel cas un conteneur redémarre-t-il ?
4. Quelle est la différence entre un réseau Docker `bridge` et `host` ?

4. Partie B – Configuration Node-RED : MQTT → InfluxDB (1h15)

Node-RED s'ouvre dans le navigateur (<http://localhost:1880>). L'interface est un éditeur de flux visuel par glisser-déposer.

4.1 Installation des nœuds InfluxDB

Dans Node-RED, cliquez sur le menu hamburger (≡) → Manage palette → Install :

- Rechercher et installer : `node-red-contrib-influxdb`
- Redémarrer Node-RED si demandé

4.2 Flux Node-RED – MQTT vers InfluxDB

Construisez le flux suivant par glisser-déposer depuis la palette gauche :

```

[mqtt in] → [function: parser JSON] → [influxdb out]
      |
      └─→ [debug: afficher payload] ───┘

```

Nœuds à utiliser :

- `mqtt in` : catégorie Network
- `function` : catégorie Function
- `influxdb out` : catégorie Storage (installé ci-dessus)
- `debug` : catégorie Common

4.3 Configuration du nœud MQTT In

Champ	Valeur
Server	mosquitto (nouveau serveur → host: mosquitto, port: 1883)
Topic	ufhb/+capteurs/# (# capture tous les sous-topics)
QoS	1
Output	auto-detect (string → Number si possible)

Nom du host 'mosquitto' et non 'localhost'

Dans Docker, les conteneurs communiquent via leur nom de service (résolution DNS interne). Node-RED doit utiliser 'mosquitto' comme hostname — pas 'localhost' ni l'IP. C'est la résolution automatique fournie par le réseau Docker bridge `iot-network`.

4.4 Configuration du nœud Function – Parser JSON

Double-cliquez sur le nœud Function et collez le code suivant :

```
// Extraction du type de mesure depuis le topic
// Topic format: ufhb/<login>/capteurs/<type>
const topicParts = msg.topic.split('/');
const typeMesure = topicParts[topicParts.length - 1]; // temperature, humidite,
luminosite
const login      = topicParts[1];

// Valeur numérique
const valeur = parseFloat(msg.payload);

// Format attendu par node-red-contrib-influxdb
msg.payload = [
  {
    measurement: typeMesure,      // Nom de la mesure InfluxDB
    tags: {
      capteur : login,           // Tag : identifiant du capteur
      site    : 'campus_ufhb',
    },
    fields: {
      valeur: valeur,            // Champ numérique obligatoire
    },
    timestamp: new Date(),        // Horodatage (optionnel, InfluxDB le génère)
  }
];

return msg;
```

4.5 Configuration du nœud InfluxDB Out

Champ	Valeur
Server	Nouveau serveur → Version: 2.x, URL: <code>http://influxdb:8086</code>
Token	<code>mon-token-secret-ufhb-2025</code> (défini dans <code>influxdb/.env</code>)
Organisation	ufhb
Bucket	capteurs_iot

Cliquez sur Deploy (bouton rouge en haut à droite). Le flux est maintenant actif.

Test du flux

Pour tester sans l'ESP32, utilisez MQTTX pour publier manuellement :

```
Topic   : ufhb/test_binome/capteurs/temperature
Payload : 28.5
QoS     : 1
```

→ Le nœud debug de Node-RED doit afficher le payload reçu.
→ InfluxDB doit recevoir le point (vérifiable via Data Explorer).

Questions de compréhension – Partie B

5. Pourquoi le nœud Function reçoit-il le topic en plus du payload ?
6. Qu'est-ce qu'un 'measurement' dans InfluxDB ? Comparer avec une table SQL.
7. Pourquoi séparer tags et fields dans InfluxDB ?

5. Partie C – Grafana : Dashboard et alertes (1h)

5.1 Connexion à InfluxDB depuis Grafana

Dans Grafana (<http://localhost:3000>) :

8. Menu gauche → Connections → Data Sources → Add data source → InfluxDB
9. Query Language : Flux (pas InfluxQL)
10. URL : <http://influxdb:8086>
11. Organization : ufhb
12. Token : mon-token-secret-ufhb-2025
13. Default Bucket : capteurs_iot
14. Cliquer Save & Test → doit afficher 'datasource is working'

5.2 Premier dashboard – Température en temps réel

15. Menu → Dashboards → New → New Dashboard → Add visualization
16. Sélectionner la datasource InfluxDB créée
17. Dans l'éditeur Flux, entrer la requête suivante :

```
from(bucket: "capteurs_iot")
  |> range(start: -1h)
  |> filter(fn: (r) => r._measurement == "temperature")
  |> filter(fn: (r) => r._field == "valeur")
  |> aggregateWindow(every: 30s, fn: mean, createEmpty: false)
  |> yield(name: "temperature_moy")
```

18. Type de visualisation : Time series
19. Titre : Température – Campus UFHB
20. Répéter pour l'humidité et la luminosité

21. Sauvegarder le dashboard : TP5B – Monitoring IoT

5.3 Configuration d'une alerte Grafana

Dans le panneau Température → Edit → Alert (onglet en haut) :

Champ	Valeur
Condition	WHEN last() OF temperature IS ABOVE 35
Evaluate every	30s For 1m (alerte si dépassement pendant 1 minute)
No Data	Keep Last State
Message	ALERTE : température dépasse 35°C sur campus UFHB

Manipulation – provoquer une alerte

Dans Wokwi (TP5A), modifiez la valeur simulée du DHT22 pour forcer $T > 35^{\circ}\text{C}$.

Observez dans Grafana : le panneau passe en rouge après ~1 minute.

Vérifiez l'historique des alertes : Menu → Alerting → Alert rules.

Questions de compréhension – Partie C

22. Quelle est la différence entre une alerte Grafana et une alerte Edge (TP5A) ?
23. Dans quel cas l'alerte Grafana est-elle préférable ? Dans quel cas l'alerte Edge ?
24. Expliquer le rôle de aggregateWindow dans la requête Flux.

6. Option – Connexion de l'ESP32 Wokwi au pipeline local

Pour connecter l'ESP32 Wokwi à votre Mosquitto local, deux approches selon l'environnement :

- A) Wokwi Desktop + ngrok : exposer le port 1883 via tunnel sécurisé
- B) Wokwi CLI + réseau local : l'ESP32 se connecte directement à l'IP de la machine

6.1 Modification du code ESP32 (une seule ligne)

```
// Avant (broker public HiveMQ) :
const char* BROKER = "broker.hivemq.com";

// Après (Mosquitto local) :
// Remplacer par l'IP locale de votre machine (ipconfig / ip addr)
const char* BROKER = "192.168.1.xxx"; // ← adapter

// Si Wokwi Desktop est utilisé avec connexion pont réseau,
// l'ESP32 simulé atteint le port 1883 sur l'hôte.
```

Si la connexion réussit, les données Wokwi apparaîtront directement dans votre Grafana — pipeline complet fonctionnel !

7. Livrables et grille d'évaluation

Critère	Indicateur de réussite	Pts	Évaluation
docker compose up réussi	Les 4 services sont en état running	4	Capture docker compose ps
Mosquitto opérationnel	Test publish MQTTX → message reçu par Node-RED	4	Capture debug Node-RED
Flux Node-RED déployé	Données insérées dans InfluxDB (Data Explorer)	6	Capture InfluxDB
Dashboard Grafana	3 panneaux (T°, Humidité, Luminosité) avec données	6	Capture dashboard
Alerte Grafana configurée	Règle d'alerte active, déclenchement démontré	5	Capture + démonstration
Questions Partie A	4 réponses correctes	3	Fiche rendue
Questions Partie B + C	6 réponses correctes	4	Fiche rendue
Code propre	docker-compose.yml structuré, commenté	3	Fichier fourni
Option : ESP32 → pipeline	Données Wokwi visibles dans Grafana	+5 bonus	Démonstration
TOTAL		35 pts	

Modalités de rendu (binôme)

Avant la fin de la séance, transmettre à l'enseignant :

1. Dossier iot-pipeline/ compressé en .zip (docker-compose.yml + configs)
2. Export du flux Node-RED (≡ → Export → JSON)
3. Capture du dashboard Grafana avec données réelles
4. Capture de l'alerte déclenchée (Alerting → Alert rules)
5. Fiche réponses (questions A, B, C)

Vers le Projet ECUE

Ces deux TP (5A + 5B) constituent la base technique du projet ECUE partagé L2/L3.

Dans le projet, vous devrez :

- Choisir un cas d'usage (monitoring environnemental, supervision réseau, flotte connectée)
- Déployer ce pipeline avec des capteurs réels ou simulés
- Sécuriser le broker MQTT avec TLS/X.509 (L3)
- Exposer une API REST pour la supervision externe (L3)
- Rédiger la documentation réseau du projet (co-modulation Gestion de Projet Réseau)